

Comparison Based Sorting

Lec14

Lower Bound on Efficiency

Can we do better than mergesort?

- We have seen mergesort runs in $O(n \log n)$
 - But can we do even better?
 - We will see, with some caveats, that the answer is no
- Note this is asking whether ANY algorithm might do better
 - Hence, just listing more algorithms is not going to answer the question
 - We must take a higher-level view of what sorting algorithms need to do
 - Finding lower bounds , “no algorithm can do better than” is much harder
 - Sorting is a one of the rare cases that a useful lower bound can be found – and relatively easily

Comparison-based sorting

- If a sorting algorithm only obtains any information about the elements by doing pairwise comparisons between the elements, then it is said to be “comparison-based”
- All the algorithms we have seen so far are comparison based
 - If you inspect the code, then the decision as to swaps etc. is based only on tests of the form

“if ($A[i] \leq A[j]$) ...”

- Note: Not all sorts are comparison-based:
 - Bucket/radix sort: used the actual values to rearrange the items
 - These can runs in $O(n)$, but relies on knowing the range of values in the sequence (e.g. “integers between 1 and 1000”).
 - You should know these exist, but do not need to know them

Lower bound for comparison sort

- We use an “information theoretic” argument
- We will model sorting as making comparisons between elements and obtain an implicit binary decision tree.
- At each node, a comparison between two elements is made
 - there are two possible outcomes, and we find out some information about the correct order of items in the array.
- If we collect enough information, then we can deduce the correct order of the items in the array.

Information collecting: quick version

- For n numbers, there are $n!$ different orderings.
- To be able to sort them, we have to discover which order they have
 - Remember we cannot “see the numbers”, but can only compare them.
- Each time we ask get an answer from a comparison, then we will half the number of underlying orders that the “hidden” numbers can have
- This means we are going to have to do such “halving” until we reduce $n!$ down to 1.
 - We will need at least $\Omega(\log_2 n!)$ comparisons
 - Which is $\Omega(n \log n)$

Reminder: How many comparisons?

- This means we need to do $O(\log_2(n!))$ comparisons
- By “Stirling’s approximation”
https://en.wikipedia.org/wiki/Stirling%27s_approximation

$$\log_2(n!) = n \log_2(n) + \text{“smaller terms”}$$

- Note: you should know this approximation, but (obviously) do not need to know the proof.
- Hence $O(\log_2(n!))$ is $O(n \log n)$
- This shows that $O(n \log n)$ sorting algorithms are essentially optimal
- **Comparison-based sorting cannot do better than $O(n \log n)$**

But let’s do a longer explanation:

Information collecting: example $n=1$

- Remember “Always start with simple examples”!
- There is just one number.
- No comparisons are possible
 - We always compare different elements of the array
- But then no sorting is needed
 - If there is just one number then the array is sorted
- Maybe this example was too simple
 - But always good to check 😊

Information collecting: example $n=2$

- If there are $n=2$ numbers, then all we can do is compare them.
 - Once, we compare them, then we know the actual order
 - can move elements to the correct place
- Imagine that entries of the array are “black boxes”
 - cannot look internally and can only compare them.
 - For simplicity, we assume all elements are different and that the array is 1-based
 - All we need to do is return the ordering of the indices
 - (For clarity, switch to 1-based array)

```
if ( A[1] < A[2] )  
    return order= 12 // meaning the indices  
else  
    return order= 21
```

Information collecting: world view

- We can think of the orderings “12” and “21” as being different “possible worlds”
 - we need to ask comparisons to find which “world” we inhabit

```
if ( A[1] < A[2] )  
    return world= "12" // smallest is at index 1  
else  
    return world= "21" // smallest is at index 2
```

Once we know the world, i.e. the actual orderings of the elements, then in $O(n)$ we can swap elements to be in their correct place:

Final ordering

Once we know the “world”, i.e. the actual orderings of the elements, then in $O(n)$ we can move elements to their correct place:

E.g. $n=4$ and we found we are in world “4312”

With an auxiliary array `B` just add smallest `idx=3` first

`B.add(A[3]), B.add(A[4]), B.add(A[2]), B.add(A[1])`
and then copy `B` back to `A`

Or “in-place”: can follow a “selection sort” pattern to directly move elements to their correct place starting from the right:

1. Swap `A[1] A[4]` to put us in world “231**4**”
2. Swap `A[2] A[3]` to put us in world “21**34**”
3. Swap `A[1] A[2]` to put us in world “1**234**”

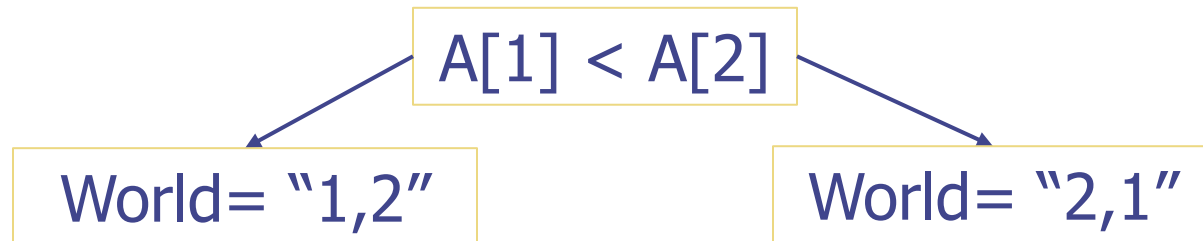
Need at most $n-1$ swaps

Information collecting: example $n=2$

```
if ( A[1] < A[2] )  
    return order= 12
```

```
else  
    return order= 21
```

Represent this as a decision tree (Yes=left)



There are $n!=2$ possible orderings: 12 and 21

- we need 1 comparison to distinguish them.

Information collecting: example $n=3$

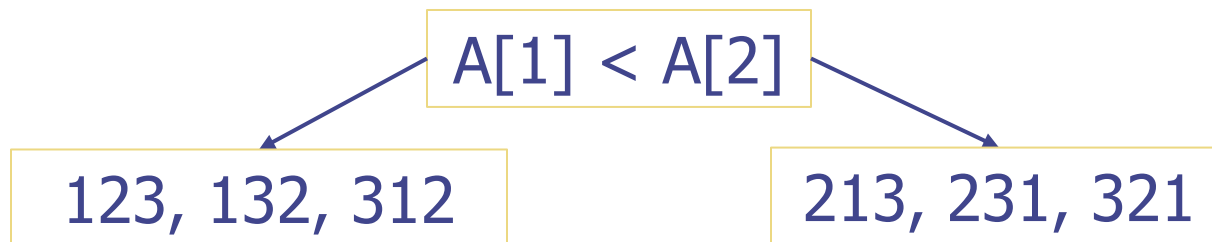
We now have $3!=6$ different orderings

123, 132, 213, 231, 312, 321

we need to ask questions $A[i] < A[j]$ until we can deduce a unique actual order

Suppose we start by asking “ $A[1] < A[2]$?”

Then list all the possible worlds that are consistent with the answer:

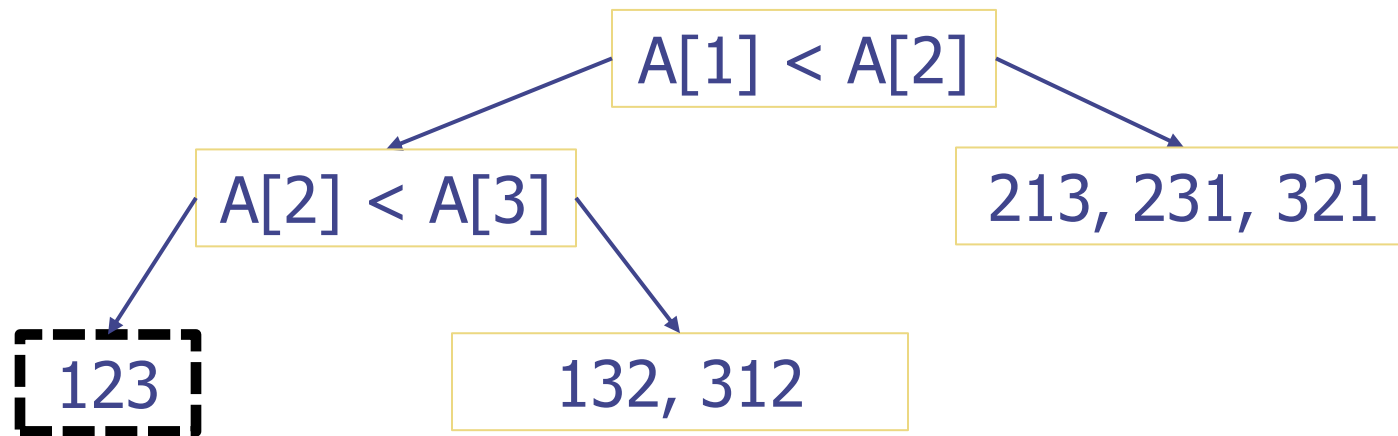


Information collecting: example $n=3$

Consider the left branch

Suppose we now ask " $A[2] < A[3]$?"

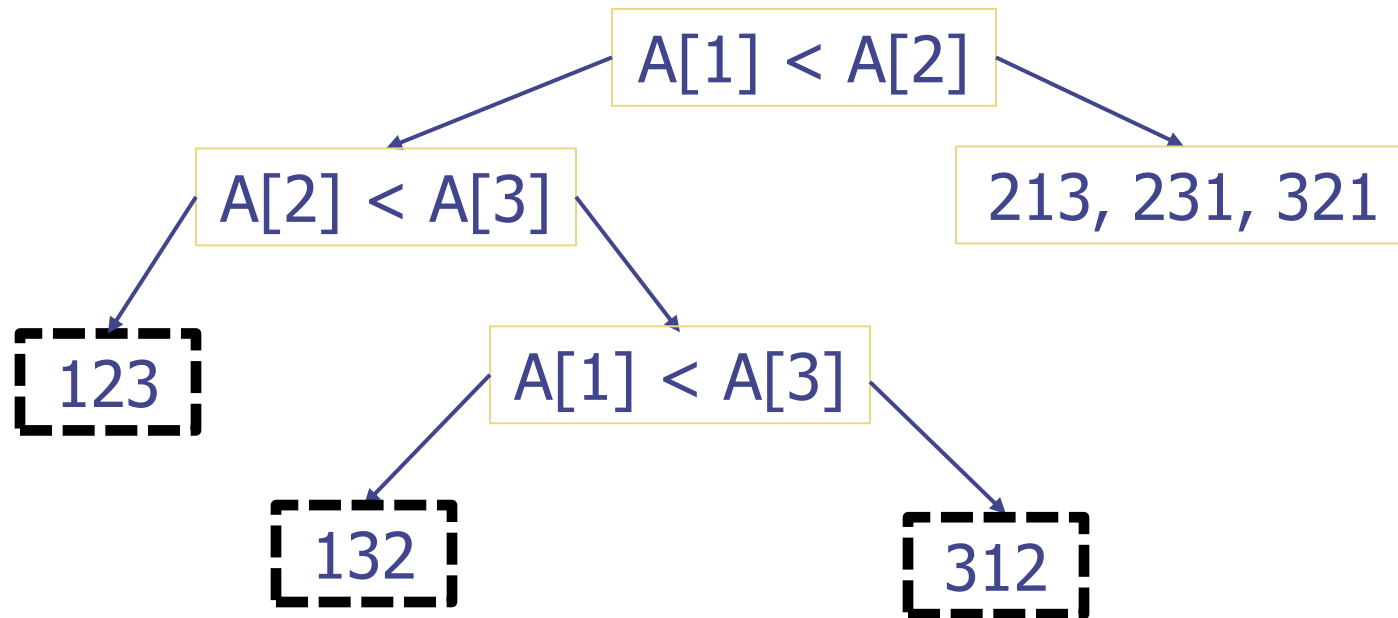
The decision tree grows and becomes:



- Once we reach a unique world then we can stop, and it becomes a leaf of the tree
- Note: if we did have $1 < 2$, and $2 < 3$ we know the world is "123" and it would be pointless to ask $1 < 3$
 - Which matches we already knew this because of transitivity

Information collecting: example $n=3$

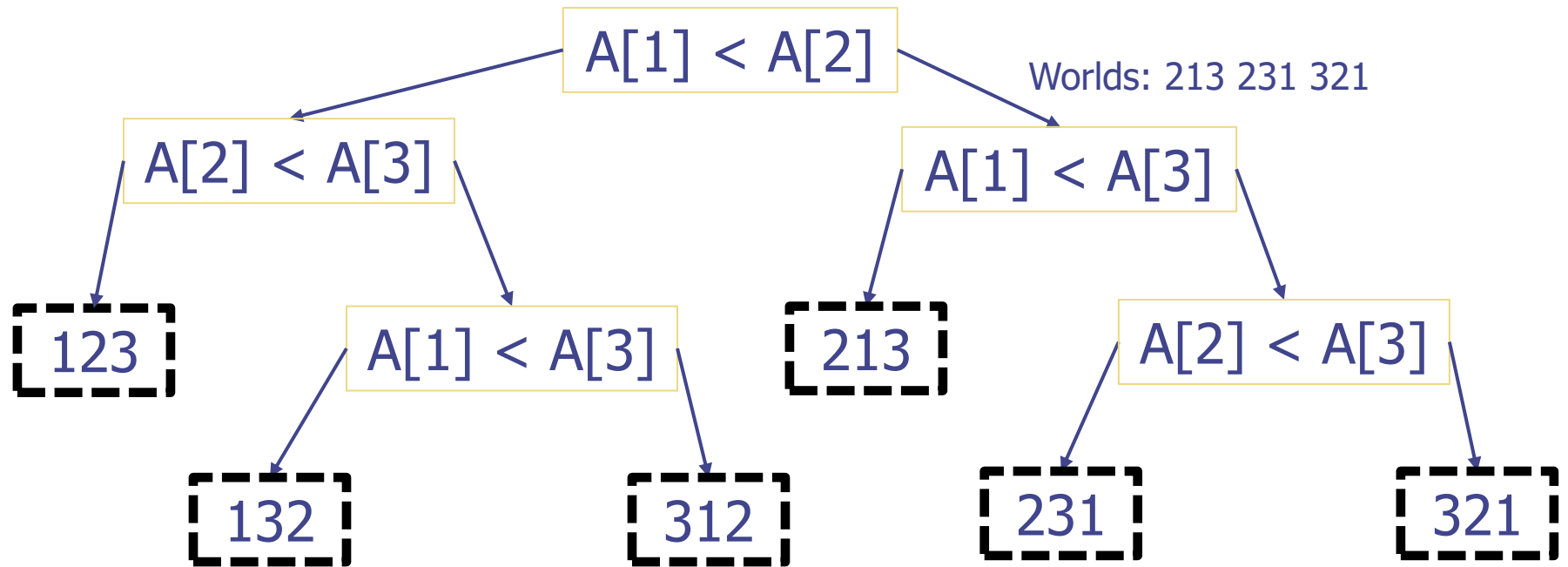
Consider the path " $1 < 2 = \text{Yes}$ ", " $2 < 3 = \text{No}$ " which left the two worlds 132 312 then we need to distinguish these. All we can do is ask " $1 < 3 ?$ "



Note: To reach worlds "132" and "312" we had to make 3 comparisons.

Information collecting: example $n=3$

Can complete the tree, e.g.



Note: we have $3!=6$ leaf nodes

The height is 3 which matches

$$\text{ceil}(\log_2(3!)) = \text{ceil}(\log_2(6)) = \log_2(8) = 3$$

Information collecting: general n

- The final decision tree is
 - Binary – as comparisons just have Yes/No answers
 - Has exactly $n!$ leaf nodes – as there are $n!$ possible “worlds” and the tree must end in a unique world
- The worst case – largest number of comparisons needed – is the number of comparison nodes on the longest path from root to a leaf node
 - The worst-case runtime is the height of the tree
- Algorithms with the best worst-case will be ones that minimise the height
- But a binary tree with $n!$ leaf nodes must have a height $\Omega(\log_2(n!))$
 - hence $\Omega(n \log n)$

Information collecting: general n

- Note that there are only $O(n^2)$ possible pairwise comparisons
 - If did all the comparisons, then we certainly can deduce the final world
 - No reasonable tree can have height worse than $O(n^2)$
 - “Reasonable” means “ignores information it has collected” or “never repeats the same question” or “never asks questions where the information would not split the set of worlds”
 - No reasonable algorithm will be worse than $O(n^2)$ comparison
 - This matches the “simple sorting” methods being $O(n^2)$
- There are unreasonable algorithms:

Bogosort

- Deterministic bogosort is

*Foreach permutation:
if (permutation is sorted)
return the permutation*

- Note the test is “permutation is sorted ?”
- Returns very little useful information when the answer is no: there is no explanation of the “no”
 - Internally, it gathers information from pairwise comparisons, but discards the information
 - Usually, we learn nothing except eliminating one world
- Takes $n!$ in the worst case
 - Poor scaling due to the poor information gathering

Minimum Expectations

- Know the meaning of 'comparison-based sorting' and the resulting $\Omega(n \log n)$ lower bound on their complexity.